Теоретический минимум

к проекту «Дообучение GPT-2-рекомендера с подкреплением по обученной модели награды»

11 октября 2026

1 О чём проект

Есть лог: кто, что и насколько полно слушал или смотрел. Проект состоит из пяти шагов.

1. Учим модель предсказывать следующий объект пользователя по его истории. Это *базовый рекомендер*.
2. Учим вторую модель предсказывать *реакцию* пользователя на объект: какую долю он досмотрит, продолжит ли сессию, вернётся ли. Это *модель награды*.
3. Дообучаем рекомендер обучением с подкреплением (RL), чтобы он выдавал списки, на которые модель награды предсказывает лучшую реакцию.
4. Смотрим, что изменилось: выросла ли предсказанная реакция, сохранилась ли релевантность, не стали ли всем показывать одно и то же.
5. Сравниваем с более простыми способами получить тот же результат.

Главная трудность в шаге 3. Модель награды отвечает на вопрос «если пользователь это посмотрит, то насколько полно?», но не на вопрос «захочет ли он это смотреть?». Рекомендер, которого награждают только её оценкой, может уйти к объектам, которые модель награды оценивает высоко, но которые пользователю не нужны. Почти вся работа о том, как это заметить и как этого не допустить.

Таблица 1. Обозначения, общие для этого документа, отчёта и ноутбуков.

Обозначение	Смысл
i_t	объект (трек, фильм), с которым пользователь взаимодействовал на шаге t
$\rho_t \in [0,1]$	доля объекта, которую пользователь прослушал или посмотрел
$c_t, u_t \in \{0,1\}$	сессия продолжилась; пользователь вернулся
h	история пользователя к моменту запроса
$\pi_\theta(j \mid h)$	рекомендер: вероятность, что следующим будет объект j; θ — его веса
π_0	рекомендер до дообучения (базовый)
$a_{1:K}$	список из K рекомендаций, порождённый объект за объектом
F	множество следующих M объектов, которые пользователь реально потребил после h
$\hat\rho, \hat c, \hat u, \hat q$	предсказания модели награды: доля, продолжение, возвращение, выбор
r_k, G_k	награда за k-й объект списка и возврат: сумма наград от шага k до конца

2 Рекомендации как продолжение последовательности

2.1 Модель следующего объекта

Историю пользователя можно читать как текст, в котором слова — это объекты каталога. Тогда рекомендация сводится к задаче языковой модели: по началу последовательности предсказать её продолжение. Модель задаёт распределение над следующим объектом,

$$\pi_\theta(i_{t+1} = j \mid i_{1:t}), \quad \sum_j \pi_\theta(j \mid i_{1:t}) = 1,$$

и обучается так, чтобы реальный следующий объект получал большую вероятность. Функция потерь — кросс-энтропия, то есть минус логарифм вероятности того, что произошло на самом деле:

$$\mathcal{L}_{\text{next}}(\theta) = -\sum_t \log \pi_\theta(i_{t+1} \mid i_{1:t}).$$

Чем увереннее модель предсказывает реальное продолжение, тем меньше потери. Если модель ничего не знает и считает все V объектов равновероятными, потери равны $\ln V$; это полезная точка отсчёта.

2.2 GPT-2 в трёх абзацах

GPT-2 — это трансформер-декодер. Каждому объекту каталога соответствует вектор (эмбеддинг); к нему прибавляется вектор позиции. Получается последовательность векторов, которую обрабатывают несколько одинаковых слоёв.

Главная операция слоя — *внимание*: вектор на позиции t обновляется как взвешенная сумма векторов других позиций, а веса модель вычисляет сама. В декодере внимание *причинное*: позиция t видит только позиции $1, ..., t$ и не видит будущего. Поэтому за один проход модель даёт предсказание следующего объекта сразу для каждой позиции последовательности, и ни одно из них не подглядывает в ответ.

На выходе для позиции t получается вектор состояния s_t, который сжато описывает историю $i_{1:t}$. Оценка объекта j — скалярное произведение s_t с эмбеддингом этого объекта; softmax превращает оценки в вероятности: $\pi_\theta(j \mid i_{1:t}) = \mathrm{softmax}_j(Es_t)$, где E — матрица эмбеддингов.

> **У нас.** Декодер из 4 слоёв ширины 256. Словарь — самые популярные объекты обучающего периода: 50 000 треков на Yambda, 7 992 тайтла на KION. Код: `gptrec/models/backbone.py`, `gptrec/models/policy.py`.

2.3 Два способа получить список

Модель предсказывает один следующий объект, а показать нужно список из K.

- **Ранжирование (top-K).** Один раз считаем вероятности всех объектов и берём K лучших.
- **Генерация (next-K).** Выбираем первый объект, дописываем его в историю, как будто пользователь его уже потребил, выбираем второй, и так K раз: $a_k = \arg\max_j \pi_\theta(j \mid h, a_{1:k-1})$.

При генерации каждый следующий выбор зависит от предыдущих, поэтому модель может строить список как целое: избегать повторов, учитывать порядок, поддерживать разнообразие. Именно поэтому RL применяют к генерации: он учит *политику*, которая принимает последовательность решений.

У генерации есть цена. При обучении модель всегда видела настоящие истории, а при генерации видит историю, дополненную собственными выборами, которых пользователь не делал. С каждым шагом вход всё дальше от того, на чём модель училась. Это называют *смещением экспозиции* (exposure bias).

3 Как измерить качество рекомендаций

3.1 Разбиение по времени

Данные делятся по одной оси времени: последние 7 дней — тест, 7 дней перед ними — валидация, всё остальное — обучение. Случайное разбиение здесь не годится: модель увидела бы будущее пользователя при обучении, и оценка оказалась бы завышенной. Каталог и вся статистика объектов считаются только по обучающему периоду.

Для каждого тестового пользователя делается один запрос в начале тестовой недели. История — его последние события до этого момента, «правда» F — его следующие $M = 20$ событий.

3.2 Метрики релевантности

Пусть $\mathrm{hit}_k = 1$, если k-й объект списка есть в F, иначе 0, и пусть $n = |F|$.

$$\mathrm{HitRate@K} = \max_{k \le K} \mathrm{hit}_k, \quad \mathrm{Recall@K} = \frac{1}{n} \sum_{k=1}^{K} \mathrm{hit}_k,$$

$$\text{DCG@K} = \sum_{k=1}^{K} \frac{\text{hit}_k}{\log_2(k+1)}, \quad \text{NDCG@K} = \frac{\text{DCG@K}}{\text{IDCG@K}},$$

$$\text{MAP@K} = \frac{1}{\min(n, K)} \sum_{k=1}^{K} \text{hit}_k \cdot \left(\frac{1}{k} \sum_{l=1}^{k} \text{hit}_l \right).$$

HitRate спрашивает, угадан ли хоть один объект. Recall — какая доля нужного попала в список. NDCG учитывает место: попадание на первой позиции стоит 1, на третьей $1/\log_2 4 = 0.5$; IDCG — значение DCG для идеального списка, поэтому NDCG лежит от 0 до 1. MAP усредняет точность списка в точках попаданий. Все метрики усредняются по пользователям.

Пример. $K = 5$, у пользователя 3 релевантных объекта, в список попали два, на позиции 2 и 4. Тогда HitRate $= 1$, Recall $= 2/3$, DCG $= 1/\log_2 3 + 1/\log_2 5 = 1.06$, IDCG $= 1 + 0.63 + 0.5 = 2.13$, NDCG $= 0.50$, MAP $= (1/2 + 2/4)/3 = 0.33$.

3.3 Понравилось и не понравилось

Правду F делим на две части по доле просмотра: *liked* — объекты с $\rho \geq 0.5$, *skipped* — с $\rho < 0.5$. Релевантность считается отдельно по каждой части. Это проверка, которой не нужна модель награды: если политика действительно научилась избегать того, что пользователи бросают, она должна терять релевантность на *skipped* и сохранять на *liked*.

3.4 Признаки коллапса

Релевантность не замечает, что всем пользователям показывают одно и то же. Для этого есть отдельные меры:

- **покрытие каталога** — доля объектов, которые попали хотя бы в один список;
- **коэффициент Джини** показов — насколько показы сосредоточены на немногих объектах;
- **доля пользователей, получивших самый частый объект**;
- **пересечение списков двух пользователей** — ожидаемая доля общих объектов.

3.5 Доверительные интервалы

Разница между двумя моделями может быть случайной. Чтобы это проверить, берётся изменение метрики для каждого пользователя, и из пользователей много раз делается выборка с возвращением (бутстрап). Разброс средних по этим выборкам даёт 95%-й интервал. Если интервал не содержит нуля, изменение значимо. Такой интервал покрывает случайность выбора пользователей, но не случайность обучения: чтобы оценить вторую, нужно обучить модель несколько раз с разными начальными значениями.

4 Модель награды

4.1 Что она предсказывает и как устроена

Модель награды — второй трансформер-декодер. На позиции t он получает объект i_t вместе с *реакцией на предыдущий объект* ρ_{t-1} (разбитой на 10 корзин). Поэтому его состояние на позиции t знает объект-кандидат i_t и всю историю с реакциями, но не знает реакции на сам i_t. Её он и предсказывает четырьмя выходами:

Таблица 2. Четыре выхода модели награды. Последние два добавлены во второй итерации.

Выход	Что предсказывает
$\hat{\rho}$	долю объекта, которую пользователь потребит
$\hat{c}$	вероятность, что сессия продолжится
$\hat{u}$	вероятность, что пользователь вернётся за новой сессией
$\hat{q}$	вероятность, что пользователь вообще выберет этот объект

Потери — бинарная кросс-энтропия. Для бинарных сигналов она обычная. Для доли ρ метка «мягкая»: событие с $\rho = 0.3$ считается на 30% положительным примером и на 70% отрицательным.

$$\mathrm{BCE}(\hat{y}, y) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y}).$$

Качество модели награды описывают три величины. *MAE* — средняя абсолютная ошибка. *AUC* — вероятность того, что модель даст более высокую оценку случайному «досмотренному» событию, чем случайному «брошенному»; 0.5 соответствует угадыванию, 1 — идеальному разделению. *Калибровка* — совпадает ли средняя предсказанная доля со средней наблюдаемой: если модель говорит 0.7, должно получаться около 0.7.

> **У нас.** AUC 0.87 на Yambda и 0.76 на KION, калибровка хорошая. Код: `gptrec/models/reward_model.py`.

4.2 Главное ограничение: модель видела только выбранное

В логе есть только те объекты, которые пользователь сам выбрал или которые ему показали и он начал потреблять. Пар «пользователь, объект, который ему не интересен» там нет. Поэтому $\hat{\rho}$ — это оценка величины

$$\mathbb{E}[\rho \mid \text{пользователь уже потребляет объект}],$$

а не реакции на произвольный объект. Для объекта, который такой пользователь никогда бы не выбрал, модель ничего не видела при обучении и подставляет то, что знает об объекте в среднем. Это *смещение отбора*.

Пример. Нишевый трек слушают 200 человек, и все они его любят: средняя доля 0.95. Модель награды запоминает «у этого трека высокая доля» и предсказывает 0.9 любому пользователю. Рекомендер, которого награждают за $\hat{\rho}$, начинает показывать этот трек всем. Именно это произошло на Yambda.

Хорошее качество на отложенных данных от этого не защищает: оно измерено на событиях того же типа, что и обучающие. RL спрашивает модель о парах, которых в логе нет.

4.3 Голова выбора

Чтобы модель награды знала, «захочет ли», её учат отличать объект, который пользователь выбрал, от объекта, который он не выбирал. Отрицательных примеров в логе нет, поэтому их создают: в каждой последовательности один объект заменяется объектом из другого события того же батча, и модель должна сказать, что это подмена. Подмены встречаются так же часто, как объекты в логе, то есть следуют распределению популярности p. Поэтому одной популярностью подмену не отличить, и модели приходится использовать историю.

$$\mathcal{L}_q = -\frac{1}{2}\mathbb{E}[\log \hat{q}(i_t \mid h_t)] - \frac{1}{2}\mathbb{E}_{j \sim p}[\log(1 - \hat{q}(j \mid h_t))].$$

У такой задачи известен оптимум:

$$\hat{q}^*(j \mid h) = \frac{P(j \mid h)}{P(j \mid h) + p(j)}.$$

Если объект для этого пользователя вероятнее, чем в среднем по всем, $\hat{q}$ выше 0.5; если менее вероятен — ниже. Произведение $\hat{q} \cdot \hat{\rho}$ читается как «ожидаемая доля просмотра, если объект, который пользователь не выбрал бы, считать за ноль».

4.4 Сигналы удержания и цензурирование

Два сигнала описывают, что пользователь сделал *после* объекта. Пусть событие произошло в момент t, а t' — моменты более поздних событий того же пользователя:

$$c = \mathbb{1}\Big[\min_{t'}(t' - t) \le g\Big], \quad u = \mathbb{1}[\exists t' : g < t' - t \le w].$$

c — сессия продолжилась: следующее событие наступило не позже, чем через промежуток g. *u — пользователь вернулся*: в пределах окна w началась новая сессия.

У таких меток есть ловушка. Если после события ничего не видно, это может значить «пользователь не вернулся», а может значить «лог закончился раньше, чем истекло окно». Второй случай называют *цензурированием*. Если считать его за «не вернулся», модель научится занижать вероятность возвращения для последних дней лога. Поэтому исход считается известным, только если всё время ожидания лежит внутри наблюдаемых данных: $t + w \leq T$. Неизвестные исходы не участвуют ни в обучении, ни в оценке.

> **У нас.** На Yambda $g = 30$ минут, $w = 1$ день. На KION в данных только даты, поэтому сессия — один календарный день ($g = 0$), а возвращение — просмотр в другой день в пределах недели ($w = 7$ дней). Код: `gptrec/data/preprocess.py`.

4.5 Две копии модели

Обучаются две модели награды на непересекающихся половинах пользователей. Первая, R_{train}, выдаёт награды во время RL. Вторая, $R_{\text{held-out}}$, в обучении рекомендера не участвует и только оценивает готовые списки. Если R_{train} говорит «награда выросла на 40%», а $R_{\text{held-out}}$ — «на 16%», разница — это ошибки первой модели, которые политика научилась использовать.

5 Обучение с подкреплением

5.1 Постановка

В RL агент взаимодействует со средой по шагам: видит состояние, выбирает действие, получает награду, и состояние меняется. Цель — набрать как можно больше награды в сумме.

Таблица 3. Словарь RL в терминах проекта. $\gamma \in (0, 1]$ — дисконт; у нас 0.99.

Понятие RL	**В нашем проекте**
Эпизод	порождение одного списка из $K = 10$ объектов
Состояние s_k	история пользователя и уже выбранные объекты: $(h, a_{1:k-1})$
Действие a_k	следующий объект списка
Политика $\pi_\theta(a \mid s)$	рекомендер: распределение над следующим объектом
Награда r_k	реакция на a_k, предсказанная R_{train}; по умолчанию $r_k = \hat{\rho}_k$
Среда	модель награды, которая играет роль пользователя
Возврат G_k	$\sum_{l \geq k} \gamma^{l-k} r_l$, сумма наград от шага k до конца списка

Настоящего пользователя в цикле обучения нет: его заменяет модель награды. Такой RL называют *модельным* (model-based). Всё, что политика узнаёт о пользователях, она узнаёт от модели награды, включая её ошибки.

Есть тонкость: чтобы оценить k-й объект, модели награды нужна реакция на предыдущие, а для порождённых объектов реакции никто не наблюдал. Модель подставляет собственное ожидание $\hat{\rho}$. Это допущение заметно влияет на результат (раздел 6).

5.2 Градиент политики

Обозначим траекторию $\tau = (s_1, a_1, ..., s_K, a_K)$ и её суммарную награду $G(\tau)$. Цель —

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)] = \sum_\tau P_\theta(\tau) G(\tau).$$

Продифференцировать это напрямую мешает то, что от θ зависит само распределение траекторий. Помогает тождество $\nabla P = P \nabla \log P$:

$$\nabla_\theta J = \sum_\tau P_\theta(\tau) \nabla_\theta \log P_\theta(\tau) G(\tau) = \mathbb{E}_\tau[\nabla_\theta \log P_\theta(\tau) \cdot G(\tau)].$$

Вероятность траектории — произведение вероятностей действий и переходов среды; от θ зависят только первые. Кроме того, действие на шаге k не может повлиять на награды, полученные раньше. Получается

$$\nabla_\theta J = \mathbb{E}\left[\sum_{k=1}^{K} \nabla_\theta \log \pi_\theta(a_k \mid s_k) \cdot G_k\right].$$

Смысл простой: сыграли эпизод, и вероятность каждого сделанного действия увеличиваем пропорционально тому, сколько награды за ним последовало. Алгоритм, который оценивает это ожидание по выборке траекторий, называется **REINFORCE**.

5.3 Базовая линия и преимущество

Оценка по выборке шумная. Если все награды положительны, REINFORCE повышает вероятность *всех* сделанных действий, просто хороших сильнее. Шум уменьшают, вычитая из G_k *базовую линию* $b(s_k)$ — ожидаемый возврат из этого состояния. Математическое ожидание градиента при этом не меняется, потому что $\mathbb{E}_a[\nabla \log \pi_\theta(a \mid s)] = 0$. Разность

$$A_k = G_k - b(s_k)$$

называется *преимуществом*: насколько действие оказалось лучше, чем в среднем из этого состояния. Теперь вероятность действий «лучше среднего» растёт, а «хуже среднего» — падает.

> **У нас.** В первых двух запусках базовой линией был средний возврат других списков для той же истории. Это REINFORCE с базовой линией.

5.4 Критик и обобщённая оценка преимущества

Базовую линию можно выучить отдельной моделью $V_\varphi(s)$ — *критиком*, который предсказывает возврат из состояния. С ним преимущество можно оценить по одному шагу, через *TD-остаток*:

$$\delta_k = r_k + \gamma V_\varphi(s_{k+1}) - V_\varphi(s_k).$$

Если критик точен, δ_k — малошумная оценка преимущества; если ошибается, она смещена. *Обобщённая оценка преимущества* (GAE) смешивает остатки нескольких шагов:

$$A_k = \sum_{l \geq 0} (\gamma\lambda)^l \delta_{k+l}.$$

Параметр λ выбирает между смещением и шумом. При $\lambda = 0$ остаётся один шаг и полное доверие критику; при $\lambda = 1$ получается $G_k - V_\varphi(s_k)$, то есть без доверия критику, но с полным шумом возврата.

> **У нас.** Критик — копия трансформера рекомендера с одним числовым выходом; $\lambda = 0.9$. Перед обновлением преимущества приводятся к нулевому среднему и единичной дисперсии внутри батча. Код: `gptrec/training/rl.py`.

5.5 PPO

У градиента политики есть практическая проблема: один слишком большой шаг может испортить политику, и следующие данные будут собраны уже плохой политикой. PPO (Proximal Policy Optimization) ограничивает шаг. Пусть w_k — отношение вероятностей действия при новых и старых весах:

$$w_k = \frac{\pi_\theta(a_k \mid s_k)}{\pi_{\theta_{\text{old}}}(a_k \mid s_k)}, \quad \mathcal{L}_{\text{PPO}} = -\mathbb{E}[\min(w_k A_k, \text{clip}(w_k, 1-\varepsilon, 1+\varepsilon) A_k)].$$

Пока w_k близко к 1, это обычный градиент политики. Когда вероятность действия уже изменилась больше, чем на долю ε, в выгодную сторону, дальнейшее изменение не поощряется.

Благодаря этому по одному батчу траекторий можно сделать несколько шагов градиента, и данные используются эффективнее.

> **У нас.** $\varepsilon = 0.2$, четыре шага градиента на батч из 512 списков. В запуске на KION ограничение ни разу не сработало: шаги были малы и без него. Фактически это метод «актёр-критик» с GAE.

5.6 Якорь

К потерям PPO добавляется штраф за отход от базового рекомендера:

$$\mathcal{L}(\theta) = \mathcal{L}_{\text{PPO}}(\theta) + \beta \cdot \mathbb{E}[\text{KL}(\pi_\theta(\cdot \mid s_k) \parallel \pi_0(\cdot \mid s_k))], \quad \text{KL}(p \parallel q) = \sum_j p(j) \log \frac{p(j)}{q(j)}.$$

Расхождение Кульбака — Лейблера равно нулю, когда распределения совпадают, и растёт, когда они расходятся. Вес β задаёт силу штрафа. В отчёте этот штраф называется *якорем*.

Якорь нужен потому, что в награде нет слова о релевантности. Единственный источник знаний о том, что пользователю подходит, — базовый рекомендер. Якорь говорит: «повышай награду, но не уходи далеко от того, что рекомендовал бы π_0». При $\beta = 0$ политика идёт туда, куда ведёт модель награды; при большом β почти не двигается.

> **У нас.** Одно и то же значение β в разных постановках значит разное. На Yambda при $\beta = 0.1$ REINFORCE теряет 89% NDCG@10, а PPO не теряет ничего. Преимущества нормируются, поэтому всё решает то, какая их часть — полезный сигнал: у REINFORCE с базовой линией из других списков той же истории она больше.

5.7 Одна итерация обучения целиком

1. Взять 512 случайных моментов из обучающих историй: пользователь и точка обрезки.
2. Для каждого породить список из 10 объектов текущей политикой, выбирая объекты случайно согласно π_θ.
3. Моделью R_{train} предсказать реакцию на каждый объект каждого списка: это награды r_k.
4. Критиком оценить состояния и вычислить преимущества A_k по GAE.
5. Сделать четыре шага градиента по $\mathcal{L}_{\text{PPO}} + \beta \cdot \text{KL}$ для политики и по квадрату ошибки для критика.

Итераций — 2000 в основном запуске и 1000 в запусках перебора параметров.

6 Почему оптимизация награды ломает рекомендации

«Когда мера становится целью, она перестаёт быть хорошей мерой» — закон Гудхарта. В RL это называют взломом награды. В проекте видны три разных механизма, и их важно различать.

Использование ошибок модели. Политика находит объекты, для которых модель награды ошибается в большую сторону (пример с нишевым треком выше). Признак: R_{train} сообщает о большом росте, $R_{\text{held-out}}$ — о маленьком. На Yambda при слабом якоре: +40% против +16%, NDCG упал на 89%, один трек попал в списки 82% пользователей.

Неверно заданная награда. Модель награды права, но сама награда поощряет не то. Доля просмотра сериала считается от всего сериала, поэтому она почти всегда ниже, чем у фильма. Политика на KION перешла с сериалов на фильмы: предсказанная доля выросла на 23%, и $R_{\text{held-out}}$ это подтвердила, но предсказанное *время* просмотра упало на 11%. Ошибки модели здесь нет; ошибка в том, что мы попросили. Другой пример — награда за продолжение сессии на KION: она поощряет тайтлы, которые бросают, потому что после брошенного тайтла человек в тот же день включает следующий.

Артефакт симуляции. Награда за поздние позиции списка зависит от того, какую реакцию на предыдущие объекты подставить. На KION предсказанная доля на десятой позиции равна 0.43,

0.53 или 0.68 при трёх разных допущениях. Абсолютное значение многошаговой награды нельзя приводить без этой оговорки.

7 Как вернуть релевантность

Есть три места, куда можно поместить требование релевантности.

7.1 Снаружи награды: якорь

Это описанный выше KL-штраф. Достоинство — простота. Недостаток — политика привязана к базовому рекомендеру целиком, вместе с его недостатками, и результат резко зависит от β.

7.2 Внутри награды: награда из лога

Для моментов из обучающих историй известно, что пользователь делал дальше. Можно наградить порождённый объект за попадание в это реальное продолжение:

$$r_k = \frac{\mathbb{1}[a_k \in F]}{\log_2(k+1)}.$$

Сумма таких наград по списку равна DCG@K. Это награда из статьи Петрова и Макдональда [1]: их политика оптимизирует NDCG напрямую и добавляет к нему второе слагаемое (разнообразие или штраф за популярность). У нас вторым слагаемым служит предсказанная доля: $r_k = \hat{\rho}_k + w_F \cdot \mathbb{1}[a_k \in F]/\log_2(k+1)$. Замысел в том, что такой награде якорь не нужен: нерелевантный объект не получает её второй части.

> **У нас.** На KION замысел подтвердился наполовину. Релевантность такая награда удерживает и без якоря (NDCG@10 +0.6%), а в сумме с долей просмотра — лучше якоря при той же награде. Но списки стягиваются к популярному: покрытие каталога падает с 0.50 до 0.10. Попасть в реальное продолжение проще всего популярным объектом.

7.3 Внутри модели награды: взвешивание выбором

Награда умножается на вероятность выбора: $r_k = \hat{q}_k \cdot \hat{\rho}_k$. Объект, который пользователь не выбрал бы, получает мало, какой бы высокой ни была его средняя доля просмотра. В отличие от предыдущего способа, эта награда определена для любого объекта и любого пользователя, в том числе на тесте. Всё зависит от того, насколько хорошо голова выбора знает о релевантности.

> **У нас.** На KION это не сработало. Голова выбора отличает подмену (AUC 0.70), но среди кандидатов рекомендера знает о релевантности гораздо меньше самого рекомендера (0.56 против 0.70), и политика с такой наградой теряет релевантность быстрее, чем с простой долей. Вероятная причина: подмены берутся по популярности, поэтому $\hat{q}$ штрафует популярное, а на KION популярное часто и есть релевантное.

7.4 Не покидать лог: REINFORCE вне политики

Статья Матвеева и соавторов [2] идёт другим путём: политика ничего не порождает, а учится на действиях, которые уже есть в логе, с наградами, которые реально наблюдались. Трудность в том, что лог собран другой политикой — *поведенческой* π_b. Её исправляют *выборкой по значимости*:

$$\mathbb{E}_{a\sim\pi}[f(a)] = \mathbb{E}_{a\sim\pi_b}\left[\frac{\pi(a)}{\pi_b(a)} f(a)\right].$$

Каждое залогированное действие взвешивается тем, насколько оно вероятнее при новой политике, чем при поведенческой. Веса могут быть очень большими, и оценка становится шумной, поэтому их ограничивают сверху порогом κ. Потери:

$$\mathcal{L}(\theta) = -\frac{\sum_t \gamma^{\tau_t} \bar{\omega}_t G_t \log \pi_\theta(i_t \mid i_{1:t-1})}{\sum_t \gamma^{\tau_t} \bar{\omega}_t}, \quad \bar{\omega}_t = \min\left(\kappa,\ \prod_{l=t}^{t+L-1} \frac{\pi_\theta(i_l \mid i_{1:l-1})}{\pi_0(i_l \mid i_{1:l-1})}\right).$$

Здесь G_t — дисконтированная сумма *залогированных* наград до конца сессии, τ_t — номер шага в сессии. Проще говоря, это обычное обучение на следующий объект, в котором события с хорошим продолжением весят больше. Такая политика не может сойтись к десяти объектам для всех: она лишь перевзвешивает то, что люди реально делали. Плата — она не может найти ничего, чего в логе нет.

> **У нас.** На KION при пороге $\kappa = 0.5$ метод даёт $+17\%$ награды при прежней релевантности на досмотренном и без потери разнообразия. С ростом порога релевантность быстро теряется: при $\kappa = 10$ — половина NDCG@10 ради $+26\%$ награды.

8 Нужен ли вообще RL

RL сложен и дорог, поэтому его нужно сравнивать с простыми способами поднять ту же награду.

- **Дообучение только на понравившемся.** Обычное обучение на следующий объект, но только по событиям с $\rho \geq 0.5$. Модель награды не нужна.
- **Переранжирование.** Берём 100 самых вероятных кандидатов базового рекомендера, для каждого считаем $\text{score}(j) = r(j \mid h) + w \log \pi_0(j \mid h)$ и оставляем K лучших. Ничего не обучается; вес w играет роль якоря.
- **Лучший из n списков.** Порождаем n случайных списков базовым рекомендером, оцениваем каждый моделью награды и показываем лучший. Это «масштабирование во время вывода» из [2].

Когда RL должен выигрывать? Когда награда зависит от списка *как целого*: от порядка, сочетания объектов, отложенного эффекта. Если награда — сумма независимых оценок отдельных объектов, выбирать объекты по одному с высокой оценкой почти оптимально, а это и делает переранжирование.

> **У нас.** На KION переранжирование не уступает RL вплоть до роста награды примерно на треть, а дообучение на понравившемся попадает в ту же точку, что и основной запуск RL. RL уходит дальше только туда, где релевантность уже сильно потеряна. Отбор лучшего из нескольких списков слабее всех остальных способов.

9 Долгосрочные цели

Доля просмотра — мгновенная реакция. Бизнесу важнее, вернётся ли пользователь. Сигналы $\hat{c}$ и $\hat{u}$ позволяют записать награду за это:

$$r_k = \hat{q}_k^b\big(w_\rho \hat{\rho}_k + w_c \hat{c}_k + w_u \hat{u}_k\big) + w_F \frac{\mathbb{1}[a_k \in F]}{\log_2(k+1)}, \quad b \in \{0, 1\}.$$

Это общая формула награды в проекте; все варианты получаются выбором весов.

У таких наград два принципиальных ограничения.

Они наблюдательные. Модель знает: «после этого объекта в таком контексте люди чаще возвращались». Это корреляция, а не причина. Возможно, возвращаются активные пользователи, и объект тут ни при чём.

У них может не быть рычага. Если вероятность возвращения почти не различается между объектами, которые можно показать данному пользователю, то выбором объекта её не изменить. Это проверяется прямо: для 100 кандидатов каждого пользователя считается разброс сигнала *между кандидатами одного пользователя* и разброс *между пользователями*. Сигнал, который меняется только между пользователями, непригоден как награда.

Офлайн можно проверить, предсказуем ли сигнал, есть ли у него рычаг и что его оптимизация делает со списками. Вернутся ли пользователи на самом деле, покажет только онлайн-эксперимент (A/B-тест).

> **У нас.** На KION возвращение в течение недели предсказуемо (AUC 0.70–0.72), но рычага почти нет: между кандидатами одного пользователя сигнал различается на 0.03, между пользователями — на 0.13. Награда за него даёт +1.9 пункта предсказанной вероятности возвращения за 7% NDCG@10.

10 Пять вопросов задания

Таблица 4. Исследовательские вопросы и эксперименты, которые на них отвечают.

№	Вопрос	Чем отвечаем
1	Меняется ли поведение и растёт ли награда?	предсказанная реакция по R_{train} и $R_{\text{held-out}}$; какие объекты получили и потеряли показы
2	Сохраняется ли релевантность; не получают ли все одно и то же?	Recall, NDCG, MAP по *all*, *liked*, *skipped*; покрытие, пересечение списков
3	Долгосрочные цели и удержание	сигналы c и u, их предсказуемость и рычаг; награды за них
4	Генерация на несколько шагов	метрики по позициям списка; три допущения об обратной связи
5	Компромисс между наградой и релевантностью	перебор β; релевантность внутри награды; сравнение с альтернативами RL

11 Словарь и карта кода

Таблица 5. Соответствие терминов. Пути указаны относительно каталога `gptrec/`.

По-русски	В коде и ноутбуках	Где в коде
базовый рекомендер, π_0	pretrained policy	`models/policy.py`, `training/supervised.py`
модель награды	reward model: train, heldout	`models/reward_model.py`
доля просмотра, ρ	`played_ratio`	`data/preprocess.py`
продолжение сессии, c	`session_continues`	`data/preprocess.py`
возвращение, u	`returns`	`data/preprocess.py`
вероятность выбора, $\hat{q}$	`is_chosen`	`training/supervised.py`: `choice_loss`
ранжирование и генерация	top-k, next-k	`rollout.py`
награда	`RewardSpec`, `RewardFunction`	`rollout.py`
якорь, β	`reference_kl_weight`	`training/rl.py`
RL (PPO)	`finetune_with_rl`	`training/rl.py`
REINFORCE вне политики	`finetune_on_logged_returns`	`training/off_policy.py`
переранжирование	`rerank`	`rollout.py`
лучший из n списков	`best_of_sampled_lists`	`rollout.py`
метрики релевантности	`ranking_metrics`	`evaluation/relevance.py`
признаки коллапса	`recommendation_profile`	`evaluation/behavior.py`
стадии эксперимента	`Experiment`	`experiment.py`

Литература

1. A. V. Petrov, C. Macdonald. Aligning GPTRec with Beyond-Accuracy Goals with Reinforcement Learning. arXiv:2403.04875, 2024.
2. A. Matveev, S. Makeev, A. Krasilnikov, V. Baikalov, S. Liamaev, K. Khrylchenko. Session-Level Optimization for Large-Scale Retrieval using REINFORCE with Multi-Step Off-Policy Correction. arXiv:2607.02818, 2026.

3. J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.
4. J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation. arXiv:1506.02438, 2015.
5. W.-C. Kang, J. McAuley. Self-Attentive Sequential Recommendation. arXiv:1808.09781, 2018.